

OpenAI Platform

 Copy page

MCP servers

Customize ChatGPT with company knowledge by building and connecting to a custom remote MCP server.

Model Context Protocol (MCP) is an open protocol that's becoming the industry standard for extending AI models with additional tools and knowledge. You can build an MCP server on top of any data source. This guide covers how to set up a basic remote MCP server for use in ChatGPT.

How it works

Connect any of your tools—including proprietary systems—to ChatGPT deep research, allowing employees to access your company knowledge from ChatGPT. The general process is:

- 1 Build an MCP server optimized for deep research by exposing a `search` tool and a `fetch` tool.
- 2 Create a custom deep research connector in ChatGPT.
- 3 Include detailed usage instructions in the connector setup to help ChatGPT interact with your service effectively.
- 4 Test and refine the connector directly in ChatGPT.
- 5 Optionally (for ChatGPT Enterprise, Edu, or Team admins), publish your connector to the entire workspace—where it appears as an additional knowledge source in deep research.

Looking to access MCP remote servers in your application?

The guide you're reading is about creating remote MCP servers to connect to ChatGPT. For API access to a network of tools in your LLM application, learn how to [enable your model to use MCP remote servers](#).

The MCP ecosystem

We're still in the early days of the MCP ecosystem. Popular remote MCP servers today include [Cloudflare](#), [HubSpot](#), [Intercom](#), [PayPal](#), [Pipedream](#), [Plaid](#), [Shopify](#), [Stripe](#), [Square](#), [Twilio](#) and [Zapier](#). We expect many more servers—and registries making it easy to discover these servers—to

launch. The MCP protocol itself is also early, and we expect to add many more updates to our MCP tool as the protocol evolves.

You may want to understand [risks and safety information](#) before deciding to use custom remote MCP servers.

Build an MCP server

If you're not yet familiar with MCP, read an [introduction to MCP](#). You can find simple server instructions from your preferred tools and libraries. Here are some resources:

[Cloudflare](#)

[Azure Functions](#)

[Stainless](#)

Set up a basic remote server

As a starting place, use our deep research MCP server [sample app on GitHub](#). This minimal example demonstrates creating and running a remote MCP server for searching and fetching cupcake orders.

- 1 Clone [the repo](#) or copy the files into an existing repo.
- 2 Set up the server. In Python, you can run the following commands:

```
1 python -m venv env
2 source env/bin/activate
3 pip install -r requirements.txt
```



- 3 Run the server. It'll start on `http://127.0.0.1:8000` using SSE transport.

```
python sample_mcp.py
```



- 4 Update the sample files with any customization you need:

`sample_mcp.py` is the main server code

`records.json` is the cupcake order data (must be present in the same directory)

MCP servers can have a number of tools. Currently, connecting to MCP servers in ChatGPT is limited to enabling users to perform deep research. This means your MCP remote server should

resemble a search engine, with tools for search and document retrieval.

Set up search

Define a search tool. In our simple cupcake order example, the code looks like this:

```

1  def create_server():
2      mcp = FastMCP(name="Cupcake MCP", instructions="Search cupcake orders")
3
4      @mcp.tool()
5      async def search(query: str):
6          """
7          Search for cupcake orders – keyword match.
8          """
9          toks = query.lower().split()
10         ids = []
11         for r in RECORDS:
12             hay = " ".join(
13                 [
14                     r.get("title", ""),
15                     r.get("text", ""),
16                     " ".join(r.get("metadata", {}).values()),
17                 ]
18             ).lower()
19             if any(t in hay for t in toks):
20                 ids.append(r["id"])
21         return {"ids": ids}

```

Search semantics

The search semantics for defining this tool and exposing it through an MCP server are slightly different from what you may be used to. Here's our full specification:

```

1  {
2      "tools": [
3          {
4              "name": "search",
5              "description": "Searches for resources using the provided query string and returns",
6              "input_schema": {
7                  "type": "object",
8                  "properties": {
9                      "query": {"type": "string", "description": "Search query."}
10                 }
11             }
12         }
13     ]
14 }

```

```

11     "required": ["query"]
12 },
13 "output_schema": {
14     "type": "object",
15     "properties": {
16         "results": {
17             "type": "array",
18             "items": {
19                 "type": "object",
20                 "properties": {
21                     "id": {"type": "string", "description": "ID of the resource."},
22                     "title": {"type": "string", "description": "Title or headline of the"},
23                     "text": {"type": "string", "description": "Text snippet or summary f"},
24                     "url": {"type": ["string", "null"], "description": "URL of the resou"},
25                 },
26                 "required": ["id", "title", "text"]
27             }
28         }
29     },
30 },
31 "required": ["results"]
32 }
33 }
34 ]
35 }

```

Teach the model how to form valid queries

In the specification above, the `description` field is important. This is where you describe to the deep research model *how* to use this tool to construct a valid search query.

For example, you may want your remote MCP server to support complex query syntax from users, all expressed in one single query string, like this:

```
type:deals amount:gt:1000
```



To enable this kind of query syntax, your `description` field must teach the model how to form a valid query out of this user input. OpenAI relies on the description to prompt our deep research model to form a query string to call this search tool.

Use it to build expressive queries. As a real example, here's search tool description from HubSpot.

```
Purpose:\\n 1. Search for resources in the HubSpot CRM of a specific object type (only
```

Set up document retrieval

The document retrieval tool helps enable citations. In our cupcake order example, the code for document retrieval looks like this:

```

1  @mcp.tool()
2      async def fetch(id: str):
3          """
4          Fetch a cupcake order by ID.
5          """
6          if id not in LOOKUP:
7              raise ValueError("unknown id")
8          return LOOKUP[id]
9
10     return mcp

```

Here's our full specification:

```

1  {
2      "tools": [
3          {
4              "name": "fetch",
5              "description": "Retrieves detailed content for a specific resource identified by ID.",
6              "input_schema": {
7                  "type": "object",
8                  "properties": {
9                      "id": {"type": "string", "description": "ID of the resource to fetch."}
10                 },
11                 "required": ["id"]
12             },
13             "output_schema": {
14                 "type": "object",
15                 "properties": {
16                     "id": {"type": "string", "description": "ID of the resource."},
17                     "title": {"type": "string", "description": "Title or headline of the fetched resource."},
18                     "text": {"type": "string", "description": "Complete textual content of the fetched resource."},
19                     "url": {"type": ["string", "null"], "description": "URL of the resource. Optional."},
20                     "metadata": {
21                         "type": ["object", "null"],
22                         "additionalProperties": {"type": "string"},
23                         "description": "Optional metadata providing additional context."
24                     }
25                 }
26             },
27             "required": ["id", "title", "text"]
28         }
29     ]
30 }

```

```
28     }  
29   }  
30 ]  
}
```

Handle authentication

As someone building a custom remote MCP server, authorization and authentication help you protect your data. We recommend using OAuth and [dynamic client registration](#). To learn more about the protocol's authentication, read the [MCP user guide](#) or see the [authorization specification](#).

After connecting your custom remote MCP server in ChatGPT, users in your workspace will get an OAuth flow to your application.

Transport and tunneling

Your remote MCP server must be internet addressable, so if the server is hosted in your intranet, you need some form of tunneling. [ngrok](#) is one convenient tool to do this, but there are other tunneling solutions—e.g., from Cloudflare.

Testing and debugging

To test your MCP server, use the API [Playground](#) to make sure the server is reachable and that the tool list resolves as expected. You can use Playground to spot-check the server's ability to return results before running deep research on it—which is slower to iteratively refine (e.g., while refining your search tool description).

For best results, we recommend testing with OpenAI o3 or o3 mini in the Playground.

Connect your remote MCP server

- 1 Import your remote MCP servers directly in [ChatGPT settings](#).
- 2 Connect your server in the **Connectors** tab. It should now be visible in the composer > deep research tool. You may have to add the server as a source.
- 3 Test your server by running some prompts.

Risks and safety

Custom MCP servers enable you to connect your ChatGPT workspace to external applications, which allows ChatGPT to access, send and receive data, and take action in these applications. Please note that custom MCP servers are not developed or verified by OpenAI, and are third-party services that are subject to their own terms and conditions.

If you come across a malicious MCP server, please report it to security@openai.com.

Connecting to trusted servers

Be careful with which custom MCP servers you add to your ChatGPT workspace. Currently, we only support deep research with custom MCP servers in ChatGPT, meaning the only tools intended to be available within the remote MCP servers are **search** and **document retrieval**. However, risks still apply even with this narrow scope.

We recommend that you do not connect to a custom MCP server unless you know and trust the underlying application. For example, pick official servers hosted by the service providers themselves (e.g., we recommend connecting to the Stripe server hosted by Stripe themselves on mcp.stripe.com, instead of a custom Stripe MCP server hosted by a third party). Because there aren't many official remote MCP servers today, you may be tempted to use a MCP server hosted by an organization that doesn't operate that server and simply proxies requests to that service via an API. If you do this, be extra careful in doing your due diligence on these unofficial MCP servers, and only connect once you've carefully reviewed how they use your data and have verified that you can trust the server. When building and connecting to your own MCP server, double check that it's the correct server.

Malicious MCP servers may include hidden instructions (prompt injections) designed to make ChatGPT behave unexpectedly. While OpenAI has implemented built-in safeguards to help detect and block these threats, it's essential to carefully review and ensure connections are established only with trusted servers.

When connecting to MCP servers that define their own tool definitions, your organization may get requests for data that you do not want or intend to share with the host of that MCP server. Before connecting to any MCP server, review the type of data being shared carefully and robustly.

MCP servers may update tool behavior unexpectedly, potentially leading to unintended or malicious behavior.

Building servers

Be careful with the data you allow access to. Your remote MCP server permits others to connect OpenAI to your services and allows OpenAI to access, send and receive data, and take action in these services. Avoid putting any sensitive information in the JSON for your tools, and avoid storing any sensitive information from ChatGPT users accessing your remote MCP server.

As someone building an MCP server, don't put anything malicious in your tool definitions. At this time, we only support search and document retrieval.

Deploy your MCP server for others to use

Large enterprises may want to deploy an MCP server to allow others to use their company knowledge with deep research in ChatGPT. To deploy an MCP server, work with your admin.

